**Name & Surname** **Polimi ID**

## Exercise 1

A. Describe the map parallel pattern and at least one fusion optimization that can be applied to it. (1)

B. Please explain what is meant by AoS and SoA, and in which cases you would use one or the other. (1)

C. Given the following C-based code:

```
int acc = 0;

for (i = 3; i < 100; i++)
   for (j = 1; j < 100; j ++) {
      a[i][2*j] = f(a[i-3][2*j-1]);
   }

for (i = 0; i < 100; i++)
   for (j = 0; j < 100; j ++) {
      c[i][j] += a[i][2*j+1]) + b[i][j];
   }
```

Where both matrices a, b and c store integers and already contain actual data (e.g., are not full of zeros).
How and under what conditions can the previous code be parallelized? Which OMP pragmas will you use to parallelize the code? (2)

**Name & Surname** **Polimi ID**

## Exercise 2

A. Please describe the parallel gather pattern, including assumptions, PRAM complexity. In which other parallel patterns discussed during the lecture has the gather been used? (2)

B. Please describe how the Unsplit parallel pattern works and its performance considering a PRAM model. How is it implemented (pseudo code required)? (2)

**Name & Surname** **Polimi ID**

**Exercise 3**

A. If you need to parallelize a breadth-first search in a graph using OpenMP, which worksharing construct would you use? (Explain the reason) (1)

B. What does it mean that OpenMP has a relaxed memory model? (1)

C. What is the parallelization issue within the following code fragment that aims to parallelize a simple loop that initializes an array? (Explain the reason) (2)

```
int offset = 5;
int result[N];

#pragma omp parallel for default(shared) private(offset)
for (int i = 0; i < N; i++) {
   result[i] = i + offset;
}
```

**Name & Surname** **Polimi ID**

**Exercise 4**

A. Make an example where the usage of the tag parameter matters (no point for vague or generic examples) (1) **This question can be skipped if you got 1 or 3 points in the second challenge.**

B. What is the difference between MPI, Open MPI, and MPICH? (1)

C. Write a meta code that shows how you can implement a scatter operation using MPI point-to-point operations on the root and non-root processes (2) **This question can be skipped if you got 2 or 3 points in the second challenge.**

Root process meta code | Non-root processes meta code